

Diffie-Hellman Bazaar Project

Documentation

Introduction to Cybersecurity 2025-26 Homework Project
(December 2025)

Author: Kornél Stefán (TFRXIL)

Programming task 1

1 Contents

1	Contents	1
2	Project overview.....	2
3	Runtime Environment and Tooling	2
4	Core Application Architecture	3
4.1	Startup (Program.cs)	3
4.2	Services (Services/ directory).....	3
4.3	Domain Models (Models/ directory).....	3
4.4	UI Components (Components/ directory)	4
4.5	Styling	4
5	Cryptographic Workflow Walkthrough.....	4
5.1	Identification	5
5.2	Open communication.....	6
5.3	Friend Request Initiation	8
5.4	Friend Request Response	8
5.5	Friendship Confirmation	9
5.6	On-going communication	11
6	Operating the System	12
7	Source Code Tour.....	12
8	Secrets Lifecycle.....	13
9	Security Considerations and Known Limitations	13

2 Project overview

Implement the Diffie-Hellman key exchange algorithm combined with AES symmetric encryption between two communicating parties.

Diffie-Hellman Bazaar is an old school chat application built with .NET Blazor Server that demonstrates the Diffie-Hellman key exchange protocol combined with AES symmetric encryption in an open network scenario.

Users can log in with arbitrary usernames, send friend requests that negotiate shared secrets, and exchange encrypted messages and file attachments. The user interface mimics the *look and feels* of Windows XP using the XP.css framework, complete with the iconic Bliss wallpaper and taskbar.

The information exchange is handled with a simple in-memory publish/subscribe service that broadcasts messages to all connected clients. Each client manages its own set of shared secrets derived from Diffie-Hellman exchanges, allowing for encrypted communication on a per-friend basis.

The transmits trusted values such as unique sender identifier and unique encryption keys for simplification. The application uses these values for simpler implementation but in a real-world scenario, these values should be exchanged and verified differently, because an attacker could impersonate another user by sending messages with a forged sender identifier.

Every message and encryption/decryption operation is logged to the console for educational traceability.

3 Runtime Environment and Tooling

This project uses standard Dotnet solutions that works cross-platform and mixes performance with simplicity.

The solution is created with Blazor technology. Blazor is Microsoft's take on modern web technologies and paradigm. The core business logic is implemented in C# language. The user interface is created with a mixture of inline C# and HTML to provide an easily understandable flow (much like React or Angular), while having external support CSS files to support the design (sadly much like React and Angular).

Target Frameworks: Dotnet 9.0 (created before dotnet 10.0 release)

Required Toolchain

- **.NET SDKs:** .NET SDK 9 or later is required to build and run all projects. Download from <https://dotnet.microsoft.com/en-us/download>.

- **Editor or IDE:** Visual Studio 2022 17.10+, JetBrains Rider 2024+, or Visual Studio Code with the C# Dev Kit extension. The project was created inside Visual Studio.
- **Browser:** Chromium-based browsers (Edge or Chrome) are recommended for Blazor Server due to stable WebSocket support.
- **Package managers:** NuGet. All dependencies are part of the .NET SDK or served from CDNs.

Operating System

Development occurred on Windows 11 23H2. The solution is cross-platform.

Data Dependencies and Runtime Artifacts

- `.vs/` and `bin/ / obj/`: Generated build artifacts and IDE metadata. They can be cleaned at any time with `dotnet clean`.
- `wwwroot/uploads/`: The Blazor app writes encrypted or plaintext attachments to this folder using random GUID file names. The directory ships empty.

4 Core Application Architecture

The Blazor Server application separates concerns into startup configuration, services, domain models, UI components, and styling.

4.1 Startup (`Program.cs`)

- Registers Razor Components and enables interactive server rendering.
- Adds `BazaarService` and `EncryptionService` as singletons, making them shared across all connected clients.
- Configures HTTPS redirection, antiforgery protection, static asset mapping, and exception handling for production.

4.2 Services (`Services/` directory)

- `BazaarService` exposes an `OnMessageReceived` event. Messages delivered through `SendMessage` fan out to subscribers such as `Home.razor`.
- `EncryptionService` implements AES encryption/decryption using MD5 to derive the key from the shared secret. Logging records both plaintext and ciphertext for educational traceability. The AES implementation is provided by the underlying operating system.

4.3 Domain Models (`Models/` directory)

- `LoginModel`: Captures user identity and generates a random 16-byte secret per login session.

- `BazaarMessage`: Unified message envelope that can hold plaintext, encrypted payloads, attached files, or Diffie-Hellman artifacts.
- `BazaarPairRequest` and `BazaarPairResponse`: Encapsulate Diffie-Hellman math using `BigInteger.ModPow`.
- `BazaarSecretExchange`: Tracks the shared secret negotiated with each friend and the order in which friendships were created.

4.4 UI Components (`Components/` directory)

- `Home.razor`: Main interactive surface, responsible for login, message composition, friend management, and uploading attachments.
- `BazaarMessageItem.razor`: Renders individual messages and performs on-demand decryption when the viewer has the appropriate secret.
- `MessageDialog.razor`: Windows XP-styled modal for error and information dialogs.
- `TaskBar.razor`: Decorative footer replicating the Windows XP taskbar with a live clock.

4.5 Styling

- Scoped CSS files (`*.razor.css`) apply component-specific visuals, such as the Bliss wallpaper and DOS-style font for the message window.
- `wwwroot/app.css` contains base typography, form styling, and improved error boundary visuals.
- `XP.css` is loaded from a CDN to achieve the retro aesthetic without bundling extra assets.

This layered architecture keeps state inside the server process while giving developers clear entry points for experimentation or extension.

5 Cryptographic Workflow Walkthrough

The application guides users through the following steps when they establish a secure channel:

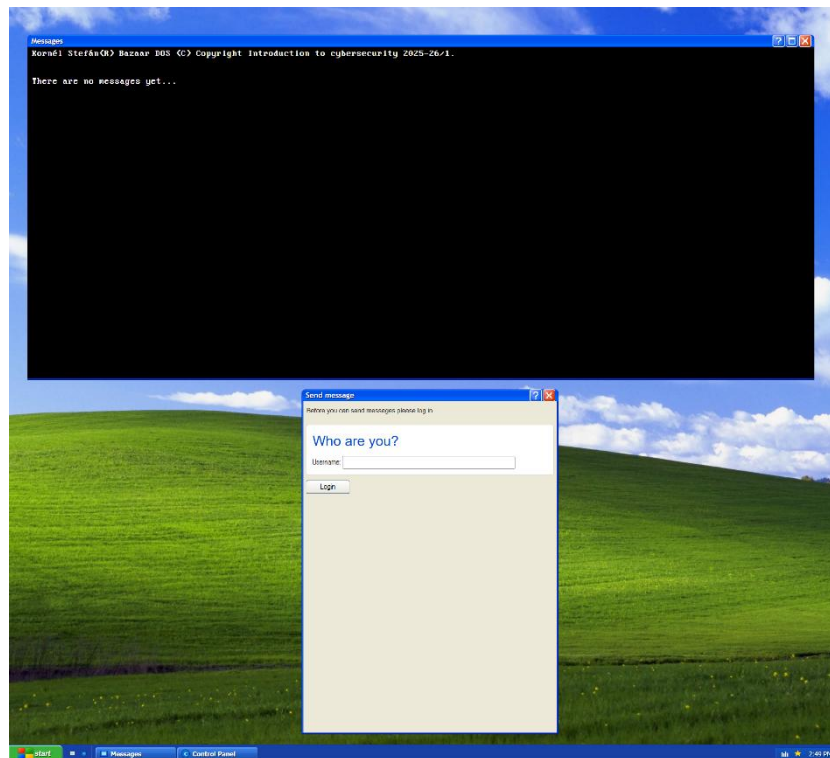


Figure 5-1 Page before login

5.1 Identification

Identification is not authentication. Anyone can claim to be anyone without higher authority making a judgement if this statement is true or false.

- The user enters an alphanumeric username.
- `LoginModel` assigns a GUID identifier and populates a 16-byte random secret (`SecretSize = 16`) using `RandomNumberGenerator.Fill` (this is a cryptographically secure source, based on the underlying operating system's implementation).
- The Settings window displays the base64-encoded identifiers so learners can differentiate “who is who”.

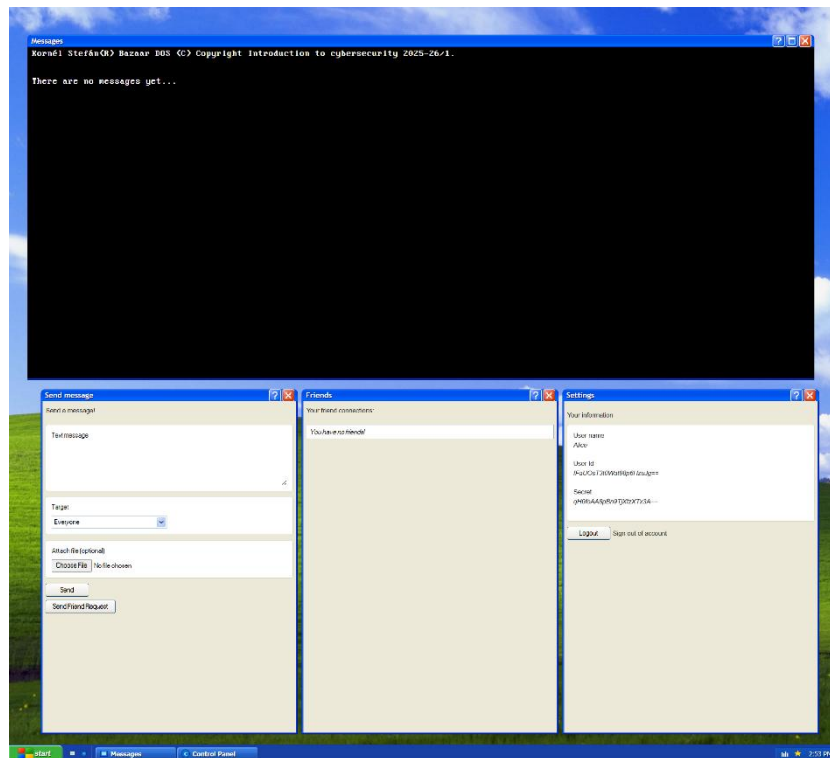


Figure 5-2 Page after logging in

5.2 Open communication

After logging in onto the page we can send messages. If there are others who have opened the page (logged in or not) they can see our message broadcasts.

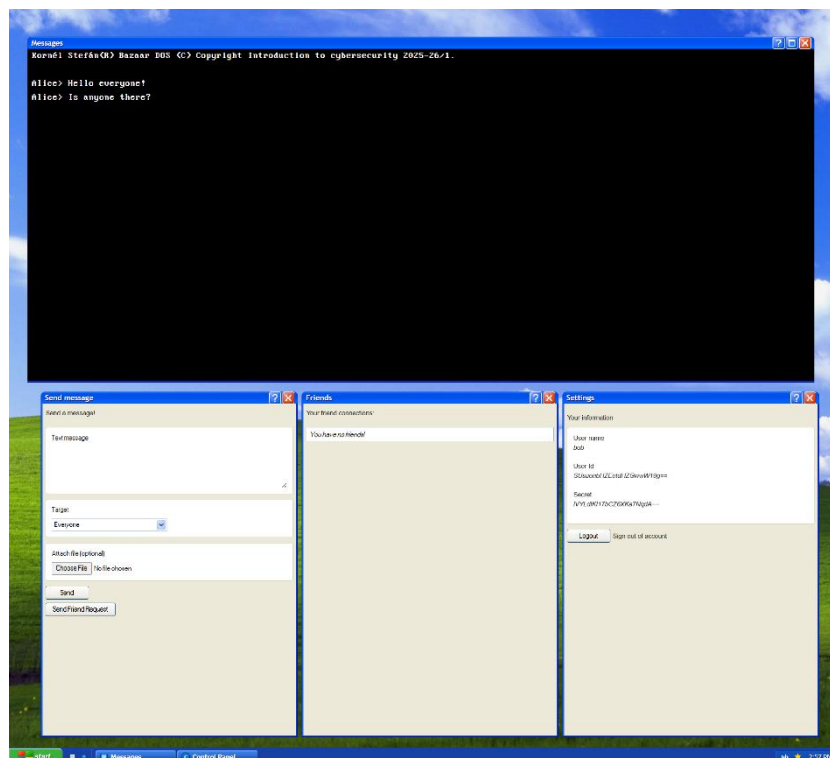


Figure 5-3 Bob receiving Alice's messages

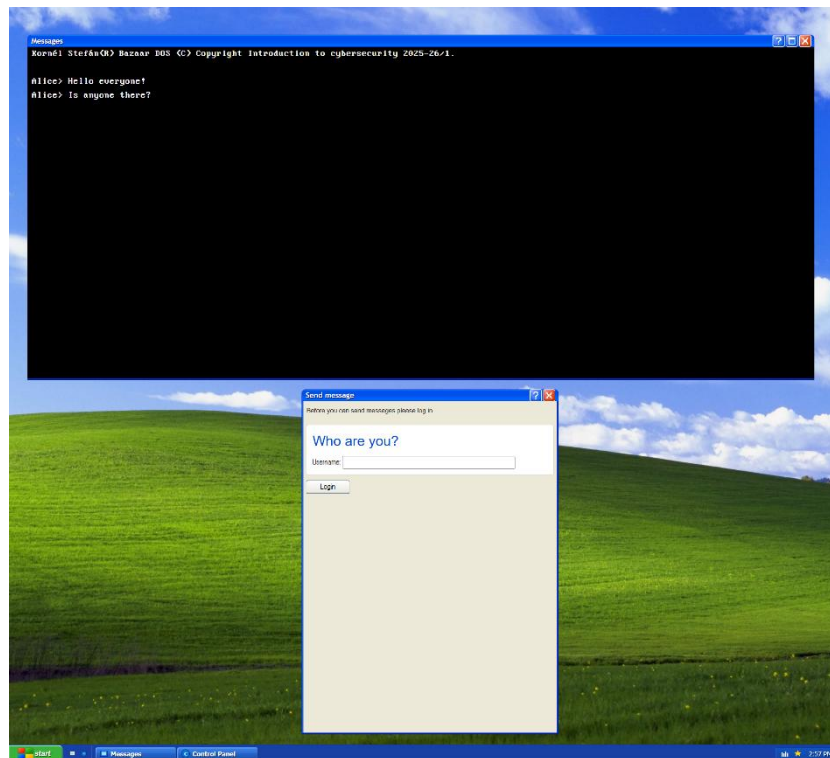


Figure 5-4 A passerby receiving Alice's messages

The others also can send responses to this message which is visible to Alice.

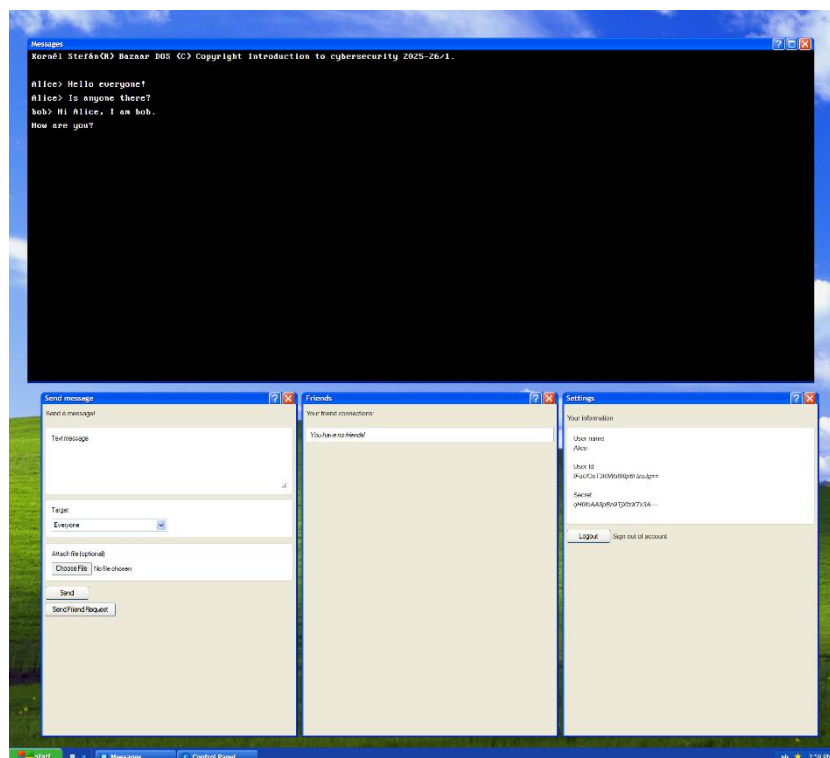


Figure 5-5 Alice receiving Bob's message

5.3 Friend Request Initiation

Alice and Bob suspect that they are not alone in this communication, but they have something private to talk about. To achieve this, they must encrypt their messages, but they don't have a shared secret. Creating a shared secret requires a trusted technique, in this case Diffie-Hellman secret sharing method. To achieve this in this application the following steps must be taken:

- Clicking "Send Friend Request" constructs a `BazaarPairRequest` based on the sender's secret `a` and sends it as an open communication message.
- Random 4-byte arrays are generated for `N` (modulus) and `G` (generator). Although small for real-world security, the size keeps calculations transparent for demonstration.
- The public component $A = g^a \bmod n$ is computed with `BigInteger.ModPow` and stored alongside `N` and `G` in the outgoing `BazaarMessage`.

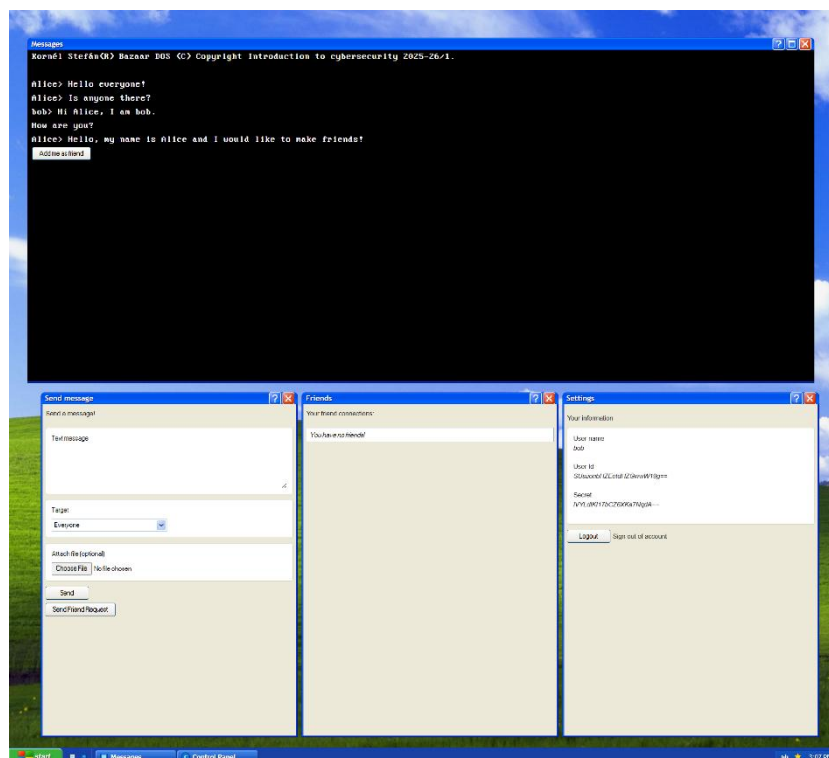


Figure 5-6 Bob receives Alice's friend request

5.4 Friend Request Response

After Bob (or anyone else) receives the friend request they can choose to respond. The response entails the following:

- The recipient clicks "Add me as friend". A `BazaarPairResponse` reuses `A`, `N`, and `G`, and computes $B = g^b \bmod n$ using their secret `b`.

- From the responder's perspective the shared secret is $S = A^b \bmod n$. The responder encrypts a greeting message with AES using `MD5(s)` as the key and a fresh IV (which is included in the encrypted message).
- The response message carries both the encrypted greeting and the full `BazaarPairResponse` structure so the initiator can compute the same secret.

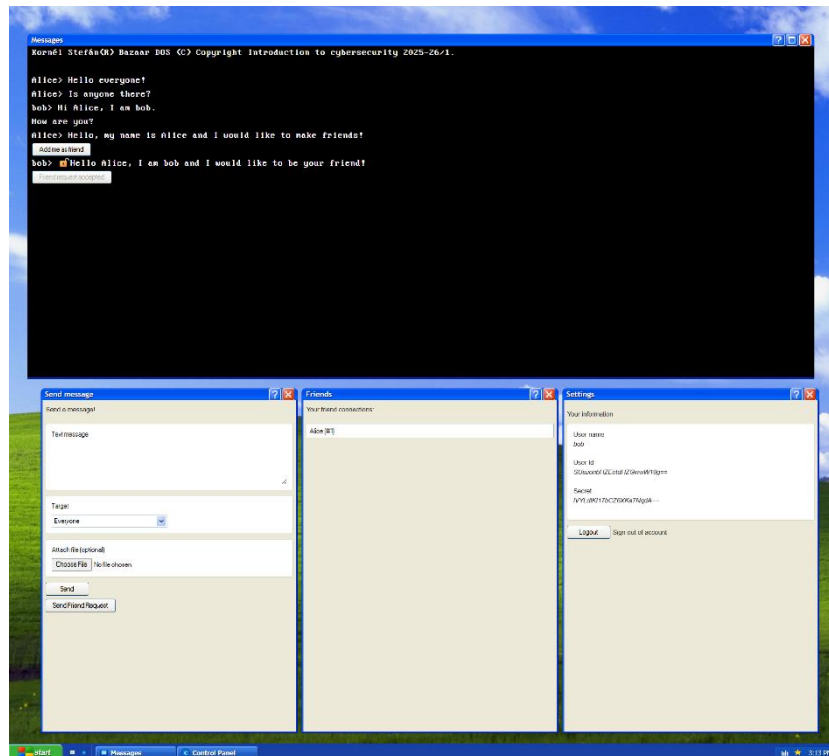


Figure 5-7 Bob responds to Alice's request.

5.5 Friendship Confirmation

Alice has received Bob's response to her friend request. Alice receives the appropriate values to finish the Diffie-Hellmann key exchange and an encrypted message. If Alice chooses to accept this, she can decrypt the message and have secure communication with (presumably) Bob.

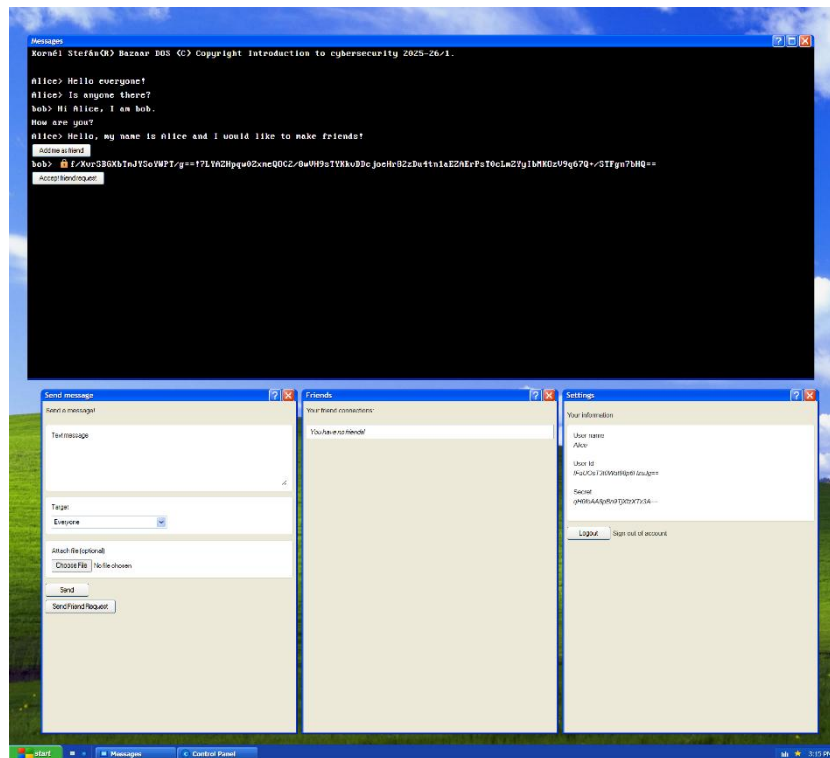


Figure 5-8 Alice sees Bob's friend request response.

- Upon receiving the encrypted response and accepting it, the initiator derives $S = B^a \bmod n$, decrypts the greeting, and records a `BazaarSecretExchange` keyed by the message identifier (simplification, in real life we don't have unique identifiers).
- The initiator sends a follow-up acceptance message encrypted with the same shared secret to confirm the channel.

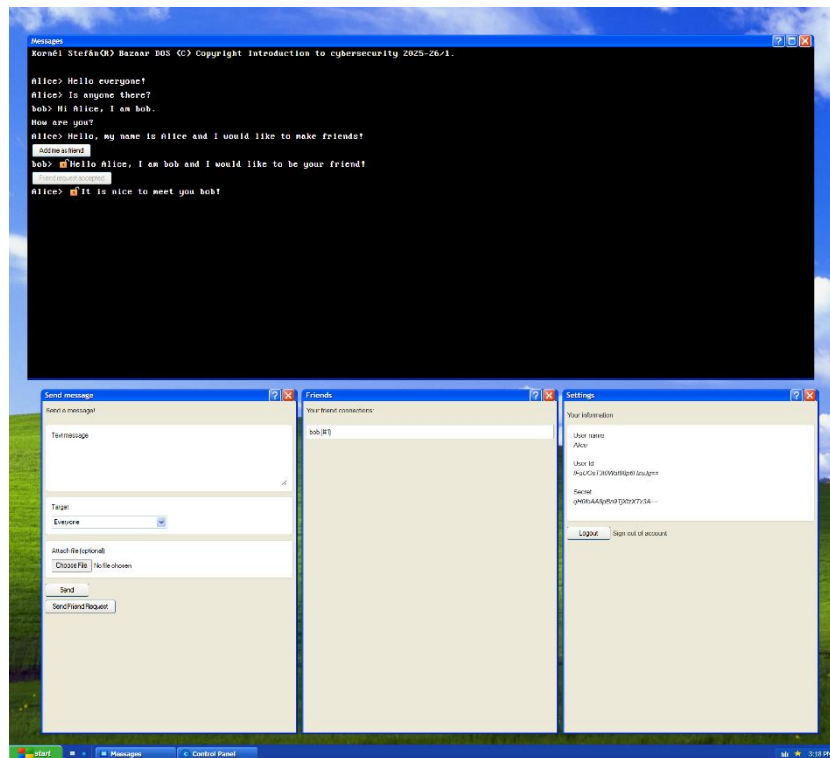


Figure 5-9 Alice accepts Bob's response.

5.6 On-going communication

Subsequent messages to that friend can now be done securely. Select the stored secret from a dropdown. Encrypting messages or attachments is opt-in; the sender can always broadcast plaintext to everyone instead.

The UI renders lock icons to indicate encryption status:  for encrypted but not decrypted,  for decryption failure, and  for successful decryption.

File attachments are read into memory, optionally encrypted, base64-encoded, and stored on disk. Receivers decrypt and download using an `IJSRuntime` bridge.

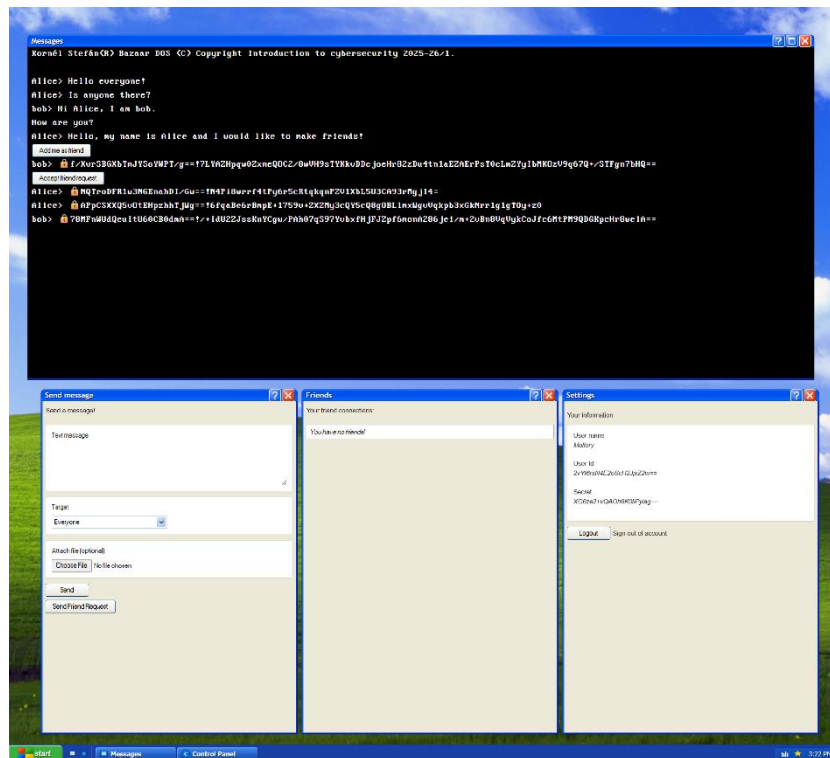


Figure 5-10 Mallory trying to spy on Alice and Bob

6 Operating the System

1. Start the Blazor server using the commands in the build section.
2. Open <https://localhost:7273> in two browser windows or two different browsers to simulate multiple users (or even more times).

7 Source Code Tour

- **Program.cs**: Configures services and middleware.
- **Services/BazaarService.cs**: Implements a simple publish/subscribe pattern. Messages are logged and relayed through the `OnMessageReceived` event. Switching to SignalR or gRPC in future work would only require replacing this service.
- **Services/EncryptionService.cs**: Centralized AES logic. It uses `MD5.Create()` purely to derive a 128-bit key from the shared secret bytes. Each encryption generates a random IV, concatenates it to the ciphertext with a `!` delimiter, and logs the transformation.
- **Models/*.cs**: Define immutable identifiers and mark required properties. For example, `BazaarMessage` stores both message content and pairing metadata.

- `Components/Pages/Home.razor`: Coordinates the user experience and application's user side logic.
- `Components/BazaarMessageItem.razor`: Is responsible for showing the incoming messages. Converts incoming encrypted strings back to plaintext when the decryptor is available. The `DownloadFile` method reads the attachment, optionally decrypts it, and streams it to the browser.
- `Components/TaskBar.razor` and `.css` companion: Provide the XP-themed taskbar at the bottom of the screen. A simple JavaScript interval updates the clock once per second.
- `Components/MessageDialog.razor`: Wraps the HTML `<dialog>` element to display user-friendly error messages with stylized SVG icons.

8 Secrets Lifecycle

- Shared secrets live only in memory within the Blazor server process. Restarting the application clears all friendships and keys.
- Attachments are stored as base64 strings in `wwwroot/uploads/{guid}`. When encrypted, the stored file remains encrypted on disk until the recipient downloads and decrypts it.
- No external secret storage or persistence layers are used, which aligns with the classroom setting and simplifies privacy compliance.

9 Security Considerations and Known Limitations

- **Small key sizes**: Diffie-Hellman parameters use 32-bit values, making the shared secret brute-forceable. The size is intentionally small for classroom visibility.
- **MD5 usage**: MD5 is cryptographically broken for collision resistance. Here it is used purely to shape the shared secret into a 128-bit AES key. Replacing it with SHA-256 and trimming the result would be more secure.
- **In-memory secrets**: All secrets vanish when the server restarts. This is acceptable for demos but limits persistence.
- **No authentication**: Users can choose any username without verification. Implementing external authentication (OAuth, OpenID Connect) would be required for real deployments.
- **Attachment storage**: Files live under `wwwroot/uploads/`. Without periodic cleanup, storage could grow indefinitely. Additionally, storing encrypted files on disk may still reveal metadata such as file size.